

SD Basic - Debugging

Stopping a program where you want it, and looking at what it thinks is true.

SD has a real source-level debugger. It stops on a line, shows you the source, prints any variable with its type, changes one, sets breakpoints, watches a variable for change, and shows the call stack. It is worth ten minutes of learning and it replaces most of the `crt` statements people leave behind.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The transcripts below are the debugger's own output, reproduced exactly as it prints them — including the `help` screen. What you see on your own screen should match character for character.

Nothing works until you compile with DEBUGGING

```
basic bp MYPROG debugging
```

The `debugging` keyword is what makes every other thing on this page possible. Without it the compiler emits no per-line debug information, and the debugger has nothing to stop on. It shows in three ways:

a <code>debug</code> statement in a program compiled without it	"WARNING: DEBUG statement ignored - Not compiling in debug mode", 0 errors , and the program ran straight through
the same program started with the <code>debug</code> verb	ran straight through — no prompt, no message
the same source compiled with <code>debugging</code>	stopped, and the prompt appeared

The warning is a warning, not an error: the program compiles and runs. **A `debug` statement in production code is therefore harmless but useless**, and the tell that you forgot the keyword is that nothing happens.

Recompile without `debugging` when you are finished. The extra information is per line, and the debugger becomes reachable by anyone who can run the program.

Two ways in

```
debug bp MYPROG          ;* the verb: stop on the FIRST line
```

The verb stops before line 1 has run:

```
:DEBUG BP ZZDBG
1:      crt 'ZZDBG.START'

>
```

```
debug                    ;* the statement: stop where you put it
```

The statement stops on the **next** line, the one that has not run yet:

```
ZZDBG.START
4:      n = n + 1

>
```

Use the verb when you want the whole program, and the statement when you want a particular place — inside an `if` that is only true for one record, for example. The statement is the only way to stop on a condition, because there is no conditional breakpoint.

`>` is the prompt. `pdebug` is the same debugger for a phantom; the `debug` verb refuses a phantom outright with *"Cannot debug program in a phantom process"*.

The commands, as this port prints them

Typing `help` at the `>` prompt gives:

```
BRK n          Set breakpoint on line n
CLR {n}        Clear all or specific breakpoint
DUMP var path  Dump variable to given pathname
HELP           Display this screen
ABORT or Quit  Abort program
EXit           Exit subroutine
EP            Exit program
```

Goto n	Goto line n
Run {n}	Free run or to line n
SRC {n}	Display line n, current line if n omitted
SRC {m,n}	Display n lines starting at line n.
STACK	Display call stack
Step .n	Step n debug elements
Step n	Step n lines
STOP	Stop program
Watch var	Watch for changes to named variable
UnWatch	Cancel Watch action

The capital letters are the abbreviation — S for Step, R for Run, G for Goto, W for Watch, UW for UnWatch, EX for Exit, Q for Quit. ? is a synonym for / below, not for help.

Looking at a variable

```
/name
```

A slash and the name, on three kinds of variable:

```
>/N
Integer: 41

>/K
String (9 bytes): "some text"

>/ARR(1)
String (5 bytes): "first"
```

The type is part of the answer, and it is often the answer. Integer: 41 and String (2 bytes): "41" are different bugs, and so are Integer: 1 and Float: 1.500000. The rest of the shapes:

a variable that has not been assigned yet

Unassigned — **not** an empty string

a name the program never mentions

Variable not defined

a floating point variable

Float: 1.500000

The difference between the first two matters: Unassigned means the variable is yours and has no value yet, and "Variable not defined" means you have mistyped the name.

A matrix element takes its subscripts: `/ARR(1)` for one dimension, `/ARR(2,3)` for two. `/` on its own repeats the last variable you looked at, printing the same answer again.

```
set name = value
```

changes it. `SET N = 100` then `/N` gives `Integer: 100`, and the program carries on with the new value.

`dump var path` writes a variable to an operating system file, which is the way to look at something too long for the screen.

Stepping and running

<code>s</code>	one line — the debugger prints the next one, <code>5: crt 'ZZDBG.N=' : n</code>
----------------	---

<code>s n</code>	n lines — <code>s 3</code> from line 4 lands on line 7
------------------	---

<code>s . n</code>	n debug elements — finer than a line
--------------------	--

<code>r</code>	run on
----------------	--------

<code>r n</code>	run on until line n
------------------	-----------------------

<code>g n</code>	jump to line n without running the lines in between
------------------	--

<code>ex</code>	run to the end of this subroutine
-----------------	-----------------------------------

<code>ep</code>	run to the end of this program
-----------------	--------------------------------

<code>stop</code>	stop the program
-------------------	------------------

<code>q</code>	abort — prints <code>ABORT : Debugging terminated</code>
----------------	--

An `s` in the middle of an assignment sequence:

```
>/N
Integer: 41

>S
5:      m = n * 2

>/N
Integer: 42
```

— the step executed line 4, `N` moved from 41 to 42, and the debugger is now sitting on line 5.

Breakpoints

```
brk 9      set one on line 9
clr 9      clear that one
clr        clear all of them
```

`brk 9` then `r`:

```
>BRK 9

>R
9:      crt 'ZZDBG.N=' : n : ' M=' : m : ' K=' : k
```

`src` shows you the line numbers to aim at. `src 1,4`:

```
>SRC 1,4
1:      crt 'ZZDBG.START'
2:      n = 41
3:      debug
4:      n = n + 1
```

Breakpoints are on a **line number in the source**, so the source record has to still match the compiled object. The debugger checks the modification time and refuses with *"Unable to find source record"* if the source has moved on; recompile rather than trying to work around it.

There is no conditional breakpoint. Put a `debug` statement inside the `if` instead, and recompile.

Watching a variable

```
w m      stop when M changes
uw       stop watching
```

A watch is the one thing that finds "who is setting this". A watch set at line 4, then `r`:

```
>W M

>R
 6:      k = 'some text'
Watched variable M has changed
Integer: 84
```

It ran on, and stopped **after** the line that changed `M`, telling you what it changed to. That is the whole answer to the commonest hard bug in a long program.

There is **one** watch at a time; `w` on another variable replaces it.

The call stack

```
stack
```

From inside a program run from the command prompt:

```
>STACK
1: /cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/ZZDBG @ 9
Command processor
```

The object path and the line, then whatever called it. From a program a program can ask the same question without the debugger:

<code>system(1002)</code>	the call stack as a dynamic array — one field per level
---------------------------	---

<code>system(1029)</code>	internal subroutine depth — <code>0</code> at the top, <code>1</code> inside a <code>gosub</code>
---------------------------	---

It is a line-mode debugger here, and that is not a limitation of your terminal

On this port the debugger never draws a full screen. Its own test is `terminfo('sreg')`, and `sreg` — *save screen region* — is a capability of SD's own client terminals, not of a console. In a session on the shipped `windows` terminal type:

<code>terminfo('sreg')</code>	0 characters
<code>terminfo('cup')</code>	16 characters
<code>terminfo('clear')</code>	6 characters

So cursor addressing is there and the region capability is not, and the debugger takes its line-oriented path. **Nothing is lost** — every command works, as the transcripts on this page show — and there is a real gain: because it is line-oriented it can be **driven from a script**. A whole debugging session can be fed down a pipe.

`@tty` reads empty in a piped session, and the debugger works there anyway.

Who has it

Read out of an account's own VOC:

VOC ENTRY	RECORD
<code>debug</code>	a <code>V</code> verb, <code>IN</code> , internal command 29 run a program in debug mode
<code>debugging</code>	a <code>K</code> keyword, 53 the compiler keyword
<code>pdebug</code>	a <code>V</code> verb, <code>CA \$PDEBUG</code> the phantom debugger

`debug` and `pdebug` are not in a standard account. Both are on the installer's `TIER.OMIT.STANDARD` list, so an account created at the standard tier has neither verb. `debugging` is a keyword rather than a verb and is unaffected — but with no `debug` verb, the `debug` **statement** is the only way in on such an account.

What is not here

`trace` is gone. It was removed from the compiler in July 2024. In an ordinary account it is *"Unrecognised statement"*.

The programmatic debug statements are restricted. `debug.on`, `debug.off`, `debug.set`, `breakpoint` and `watch` are internal-only, and all five are *"Unrecognised statement"* in an ordinary account. They are what the debugger itself is built from. `debug` — the plain statement — is not restricted, and it is the one you want.

`debug.info()` is internal-only for the same reason.

No conditional breakpoints, no data breakpoints beyond the single watch, and no remote attach. The debugger runs in the session that runs the program.

See also

[SD Basic - System and Environment](#) · [SD Basic - Program Structure](#) · [SD Basic - Program Control](#) · [SD Basic - Locks and Transactions](#).